MIT 8103 Advanced Database Systems

Real-time Game Leaderboard · Taiwo Oyewale · Student ID 301926988

Case study

My case study is Real-time Game Leaderboard. I designed the database for players submitting scores, viewers checking leaderboard rankings and operators reviewing score submissions, with separate tables named player, game and score_entry. It supports these operations: register players and games; record score submissions; filter and rank scores by game and season.

Portfolio 1: Database Design and Modelling

I organised the relational design into 3 tables. Primary keys identify records, foreign keys preserve the links, and checks enforce the stated business rules. A submitted score must be zero or greater. The normalisation examples show the sample data at 1NF, 2NF and 3NF. All 9 deliberately invalid inserts were rejected without changing the valid sample rows.

Figure 1. Tables, keys and relationships. PK: primary key; FK: foreign key; UK: unique value.

Portfolio 2: Query Processing and Optimisation

I ran three SQL queries against the sample records. The join returned 1 rows and included player_one with Sky Rally. For the index comparison, I used 10,000 separate performance rows. The median lookup time changed from 0.522 ms to 0.14 ms. Both phases returned the same records. The measured plan used the new index. These timings apply to this

lookup and environment.

Portfolio 3: Transactions and Concurrency

I used a temporary counter on score_entry to test COMMIT, ROLLBACK and READ COMMITTED isolation. The uncommitted update remained hidden from the other session, and ROLLBACK restored the earlier committed value. Without locking, two increments left the counter at 2. With SELECT FOR UPDATE, Session B waited for Session A and the final value was 3. The helper is confined to this exercise; the case's original business fields stay unchanged.

Portfolio 4: NoSQL and Advanced Data Models

I used leaderboard_events for the case's event documents. MongoDB inserted 2 sample events and returned the expected results for 4 queries. Validation rejected an invalid date type and a duplicate event ID. Embedded snapshots keep related details with an event, while the details field can differ by event type. These are fictional events; PostgreSQL continues to enforce its relational rules.

Repository and evidence

https://github.com/TaiwoSaidat/Oyewale_Taiwo_301926988
(https://github.com/TaiwoSaidat/Oyewale_Taiwo_301926988)

The portfolio folders contain the SQL and JavaScript, explanations, query plans, result tables and screenshots.

Portfolio 5: Distributed and Cloud Database Exercise

I proposed three MongoDB replica members across separate failure zones. The discussion covers primary elections, majority acknowledgement, secondary lag and sharding trade-offs for leaderboard_events. Replication supports availability, while backups address recovery. PostgreSQL needs its own availability plan. This is the permitted discussion option; no live replica set or failover test was deployed.

Individual reflection

I kept player, game, score_entry in separate tables so their details could be maintained without repeating them in every related record. A submitted score must be zero or greater. The constraint tests check this alongside the required values and record links. One issue illustrated by the two-session exercise is that both sessions can read the same value before

either update is committed. Locking the row before reading preserves both increments. The index comparison also shows why I need to check the execution plan as well as the final answer: the same result can require different amounts of database work.

AI-use declaration

I used GitHub Copilot for proof reading and the portfolio generator to help prepare the database model, scripts and explanations. The generator ran the PostgreSQL and MongoDB exercises, checked their outputs and captured the result images. I was responsible for checking and explaining the submitted work. I re-ran each portfolio's commands in a fresh environment, compared the output with the README tables, corrected the two inconsistencies I found, and checked all links and claims against the scripts

- P1: Re-ran the schema, sample data and all 9 constraint tests in a fresh database; confirmed each failed at the named constraint and the final counts stayed 1/1/1. Compared the design/ER/normalisation documents with P1_Schema.sql; confirmed PKs, FKs, unique keys and the score >= 0 check match, and that joining the 3NF tables reproduces the sample row.

- P2: Re-ran all three queries and confirmed row counts (1, 1, one group); re-ran the index experiment and confirmed the plan changed from Seq Scan to Bitmap Index Scan using p2_filter_idx. Checked that the 10,000 benchmark rows are generated in a separate query_lab table, not added to the case data.

- P3: Ran the commit/rollback sequence and confirmed uncommitted values stayed hidden and ROLLBACK restored 1; repeated the two-session test and observed Session B waiting and the final results 2 versus 3.

- P4: Re-ran the MongoDB script against a fresh portfolio_nosql database; confirmed 2 inserts, query results 1/2/2/2 groups, error codes 121 and 11000, and final count 2. Checked the date explanation: 2030 is the synthetic event time, while 2026 is the copied source timestamp.

- P5: Checked that the document clearly says "proposed only," that no live deployment is claimed, and that the architecture image path resolves.

- Documentation: Cross-checked README claims against the scripts and corrected the INTEGER/BIGINT mismatch and the "four"/"two" event comment. Followed each README's run order from a clean checkout and verified the documented expected outputs; checked that all relative links and screenshots open.

References

- PostgreSQL: constraints (https://www.postgresql.org/docs/17/ddl-constraints.html)
- PostgreSQL: EXPLAIN (https://www.postgresql.org/docs/17/using-explain.html)
- PostgreSQL: isolation (https://www.postgresql.org/docs/17/transaction-iso.html)
- PostgreSQL: row locks (https://www.postgresql.org/docs/17/explicit-locking.html)
- MongoDB: data modelling (https://www.mongodb.com/docs/manual/data-modeling/)
- MongoDB: replication (https://www.mongodb.com/docs/manual/replication/)
- MongoDB: sharding (https://www.mongodb.com/docs/manual/sharding/)

- Mermaid: ER diagrams (https://mermaid.js.org/syntax/entityRelationshipDiagram.html)